UNIVERSITY HELP DESK MANAGEMENT SYSTEM

Complete System Development Guide

Data Structures Assignment # 02 | C++ | Circular Queue | Client–Server Application

1. Assignment Overview

Develop a network-based University Help Desk Management System. Students use their own computers to submit help desk requests and check their status. Authorized help desk operators and administrators use a separate interface to view, assign, process, and update requests. Both interfaces communicate with one central application server.

The central server implements an array-based circular queue in C++ to organize pending requests in First-In, First-Out (FIFO) order. A database stores user accounts, request details, status changes, and history so records remain available after a server restart.

2. System Architecture

Component	Responsibility
Student interface	Login, submit a request, receive a request ID, view own requests and statuses.
Admin/operator interface	View pending requests, process the next request, search records, update statuses, view reports.
Central C++ application server	Validates requests, checks permissions, runs the circular queue, coordinates database operations.
Circular queue	Maintains the order of pending requests and reuses array positions through circular indexing.
Database	Stores users, requests, timestamps, request statuses, assignments, and request history.
University network (LAN)	Allows student and admin computers to access the central server.

3. Recommended Technologies

Layer	Recommended option	Purpose
Student/admin interfaces	HTML, CSS, JavaScript	Browser-based, role-specific dashboards.
Backend	C++ with a web framework such as Crow	HTTP endpoints, business logic, queue operations.
Database	SQLite for a prototype; MySQL for larger deployment	Persistent records and request history.
Network	University LAN / approved server network	Communication between clients and server.
Hosting	Central PC for demonstration; dedicated server for deployment	Runs the application for multiple users.

Why a web application? Users open a browser on their own computers, so the application does not need to be installed on every client. The C++ backend and circular queue run centrally, ensuring all users work with the same queue.

4. Student Interface Requirements

- Log in using an account approved by the university.
- Submit a request by selecting a category (registration, fee issue, examination, or other) and entering a description.
- Receive a unique request ID after successful submission.
- View only their own requests, current statuses, and previous requests.

- See clear messages if required fields are missing or the pending queue is full.

5. Administrator and Help Desk Interface Requirements

- Log in through an authorized operator or administrator account.
- View pending requests in FIFO order and inspect complete request details.
- Process the next request and assign it to an operator if needed.
- Search by request ID or student registration number.
- Update a request to Processing, Resolved, or Rejected and record remarks where appropriate.
- View counts of pending, processing, resolved, and rejected requests.
- Review request history and produce basic reports.

6. End-to-End Workflow

1. **1. Student login:** The server authenticates the user and identifies the user's role.
2. **2. Submit request:** The student interface sends request details to the C++ backend over HTTP.
3. **3. Validate:** The backend validates fields, confirms the student account, and assigns a unique request ID.
4. **4. Enqueue:** The backend inserts the request at the rear of the circular queue. If full, it returns a clear error and does not overwrite existing requests.
5. **5. Admin views queue:** The admin interface retrieves current pending requests from the server.
6. **6. Process next request:** The backend removes the front request and records that it has been assigned or is being processed.
7. **7. Update outcome:** The operator updates the request status and the server records the change in the database.
8. **8. Student checks status:** The student dashboard retrieves the latest status from the server.

Important distinction: removing a request from the pending queue means it has been assigned for processing; it does not necessarily mean the issue has been resolved.

7. Circular Queue Design

Use a fixed-capacity array of request records. For the assignment demonstration, set capacity to 5. Maintain front, rear, and count.

Variable	Meaning
front	Index of the first pending request.
rear	Index of the most recently inserted request.
count	Number of requests currently in the queue.
capacity	Maximum number of pending requests; use 5 for the required demonstration.

Circular indexing formula:

next index = (current index + 1) % capacity

Queue conditions:

- Empty: count == 0
- Full: count == capacity

The display operation must start at front and move circularly for exactly count elements. Physical array order may differ from the logical FIFO order.

8. Database Design

Table A: Users

Field	Purpose
user_id	Unique user identifier (primary key).
registration_no	Student registration number, where applicable.
name	User's name.
password_hash	Securely stored password hash; never store plain-text passwords.
role	Student, operator, or administrator.

Table B: Requests

Field	Purpose
request_id	Unique request identifier.
user_id	Student who submitted the request.
category	Registration, fee issue, examination, or other.
description	Details of the issue.
status	Pending, Processing, Resolved, or Rejected.
created_at	Submission timestamp.
updated_at	Timestamp of last update.
assigned_to	Operator responsible for handling the request, if assigned.

Table C: RequestHistory

Field	Purpose
history_id	Unique history record identifier.
request_id	Associated request.
changed_by	User who performed the action.
old_status	Previous status.
new_status	New status.
remarks	Explanation or notes.
changed_at	Timestamp of the change.

Use a database-generated primary key or another reliable method to guarantee unique request IDs. Do not rely only on an in-memory counter that resets when the server restarts.

Note: You may add more tables as per your understanding or change the design of ERD.

9. Connecting Multiple Computers

For a LAN demonstration, run the C++ server on one central computer. Configure it to listen on the appropriate network interface, permit the required port through the firewall, and make sure client computers can reach it.

Illustrative address: if the central computer has a reserved local IP of 192.168.1.50 and the application listens on port 8080, users might open http://192.168.1.50:8080 in a browser. This is only an example; use the actual address and port assigned by the network administrator.

Internet access is not required if the system is intended only for use inside the university LAN. For real deployment, use HTTPS, appropriate authentication, backups, firewall rules, and approved access controls.

10. Development Phases

9. **Phase 1 — Circular queue module:** Implement enqueue, dequeue, peek, display, isFull, isEmpty, and wraparound in C++. Test independently.
10. **Phase 2 — Request management:** Define the request structure and implement submission, search, request IDs, and status handling.
11. **Phase 3 — Database integration:** Create tables, save requests, retrieve pending requests, and preserve records after restart.

12. **Phase 4 — C++ web server:** Use a C++ web framework such as Crow to create endpoints for login, request submission, listing, and processing.
13. **Phase 5 — Web interfaces:** Build separate student and admin dashboards with HTML, CSS, and JavaScript and connect them to the backend.
14. **Phase 6 — Multi-computer testing:** Run the server on one computer and access it from at least two other computers. Test FIFO, wraparound, full queue, permissions, and concurrent submissions.

11. Testing Requirements

Test case	Expected result
Submit to an empty queue	Request is inserted successfully.
Submit five requests	Queue becomes full.
Submit a sixth request	Insertion is rejected without overwriting requests.
Process next request	Oldest pending request is selected first.
Submit after processing	Freed array position is reused correctly.
Display after wraparound	Requests appear in logical FIFO order.
Search for existing request	Correct details are displayed, subject to permissions.
Search for nonexistent request	A not-found message is displayed.
Process an empty queue	A clear error message is displayed.
Restart server	Pending requests are recovered from persistent storage.
Student tries admin action	Server denies access.
Two users submit concurrently	Queue state remains correct and no request is silently lost.

Students should submit actual outputs or screenshots and record expected versus actual results.

12. Concurrency, Security, and Reliability

- Concurrent submissions: protect queue modifications with synchronization, such as a mutex, so simultaneous requests cannot corrupt queue state.
- Multiple operators: prevent two operators from processing the same request by coordinating queue and database updates.
- Queue capacity: capacity 5 is appropriate for a classroom demonstration but too small for a real university. Keep it configurable as an extension.
- Server failure: rebuild the pending queue from the database after restart.
- Access control: students can view only their own requests; operators and administrators receive different permissions.
- Input security: validate inputs, use parameterized database queries, and hash passwords.
- Status consistency: once a request is assigned for processing, it must not remain in the pending queue.

13. Suggested Assessment Rubric

Component	Marks
Circular queue logic and FIFO behaviour	12
Request submission and processing	8
Display, search, and view-next functionality	8
Menu/interface and system statistics	5
Edge-case handling and testing	7
Code quality, modularity, and explanation	5
Flowchart or pseudocode	5
Total	50

If the web interface, database, and network components are assessed separately, increase the total marks or reallocate marks explicitly so students understand whether this is primarily a Data Structures assignment or a full-stack application project.

14. Final Student Deliverables

- Complete C++ source code for the backend and circular queue.
- Student and administrator interface source files.
- Database schema or database setup script.
- Flowchart or pseudocode for the system workflow.
- Test cases with expected and actual results.
- Screenshots showing the system running on multiple computers or browser sessions.
- A short report explaining architecture, circular indexing, role permissions, persistence, and time complexity.
- A demonstration in which students explain and defend their own implementation.

15. Final Recommendation

Treat this as a full-stack application project built around a Data Structures concept. First assess whether students correctly implement and explain the array-based circular queue. Then assess how well they integrate the queue with persistent storage, a C++ server, separate role-based interfaces, and network access. The browser interfaces should never manipulate the queue directly; all queue operations must be validated and executed by the central backend.